Abstract

This project presents a comprehensive, two-phased approach to modern cybersecurity engineering, balancing offensive forensic analysis with defensive artificial intelligence paradigms.

In the first phase, an active reconnaissance and vulnerability assessment pipeline was executed against a controlled target environment (Metasploitable 2) utilizing industry-standard network sensors and scanners including tcpdump, nmap, and Nikto. A Slowloris Denial of Service (DoS) attack was deployed to evaluate system resilience under resource depletion constraints. Subsequent forensic analysis of the resulting packet capture (.pcap) files and Apache server logs (final_access.log) successfully validated automated attack signatures, revealing distinctive traffic spikes, an anomalous ~3.5:1 HTTP 404 error ratio indicating directory enumeration, and high-density socket-exhaustion NetFlow patterns via the SiLK toolset.

In the second phase, the project shifts from reactive analysis to proactive mitigation by exploring the integration of Large Language Models (LLMs) into security infrastructure. Three cutting-edge paradigms are evaluated: LLM-based Log & Alert Analysis, LLM-Assisted Cyber Threat Intelligence (CTI) Synthesis mapped to the MITRE ATT&CK framework, and agent-based Incident Response Assistance. Finally, a strategic technology adoption roadmap, training program, and governance framework are established to safely operationalize these AI capabilities while minimizing risks such as automation bias, semantic drift, and data leakage.

Contents

Task 1: Data Collection and Analysis

Setting up the environment (Task 1-4)

For Task 1-3, I have downloaded and installed the assets (VirtualBox, Kali Linux and Metasploitable 2) that are required for me to perform data collection and analysis as stated in the project instructions.

For task 4, the tcpdump utility was initialized on the Metasploitable eth0 interface to capture fullpacket data (headers and payloads). The capture was saved to final_project.pcap for post-incident analysis.

4. Deployment of Network Sensors

A packet capture sensor was deployed on the target system (Metasploitable 2) prior to the initiation of any offensive activities to establish a baseline for network monitoring and capture traffic data for forensic analysis.

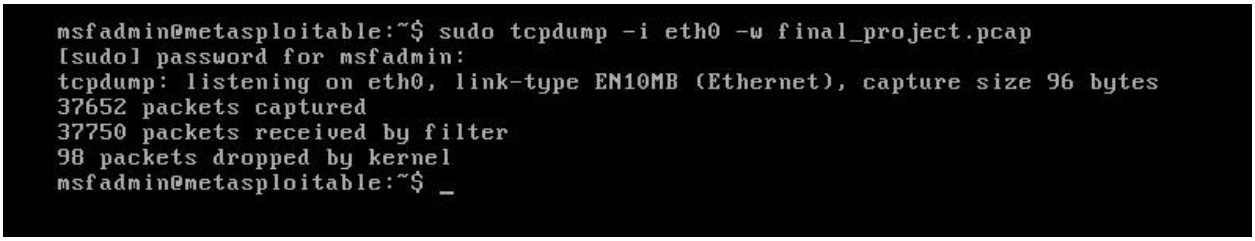


Figure 4.1: Deployment of tcpdump on Metasploitable 2 to capture full packet content.

Data Retrieval:

After completing the attack phase, the capture file and server logs were transferred to the attacker machine (Kali Linux) using the scp to ensure data integrity.

Figure 4.2: Secure transfer of the capture file (3.6MB) and access logs (5.4MB) to the analysis station.

5. Active Reconnaissance and Vulnerability Scanning

The target 192.168.194.130 was subjected to a number of active reconnaissance techniques in order to find open ports, operating system information, and application vulnerabilities. In order to replicate genuine, low-noise attacker behavior, strict timing procedures were followed, with a minimum 20-minute delay enforced between scan stages.

5.1 TCP SYN Scan

Command:

sudo nmap -sS 192.168.194.130

A TCP SYN scan was initiated to map open ports without completing the TCP. The -sS flag was explicitly used to validate the specific scan type requirement.

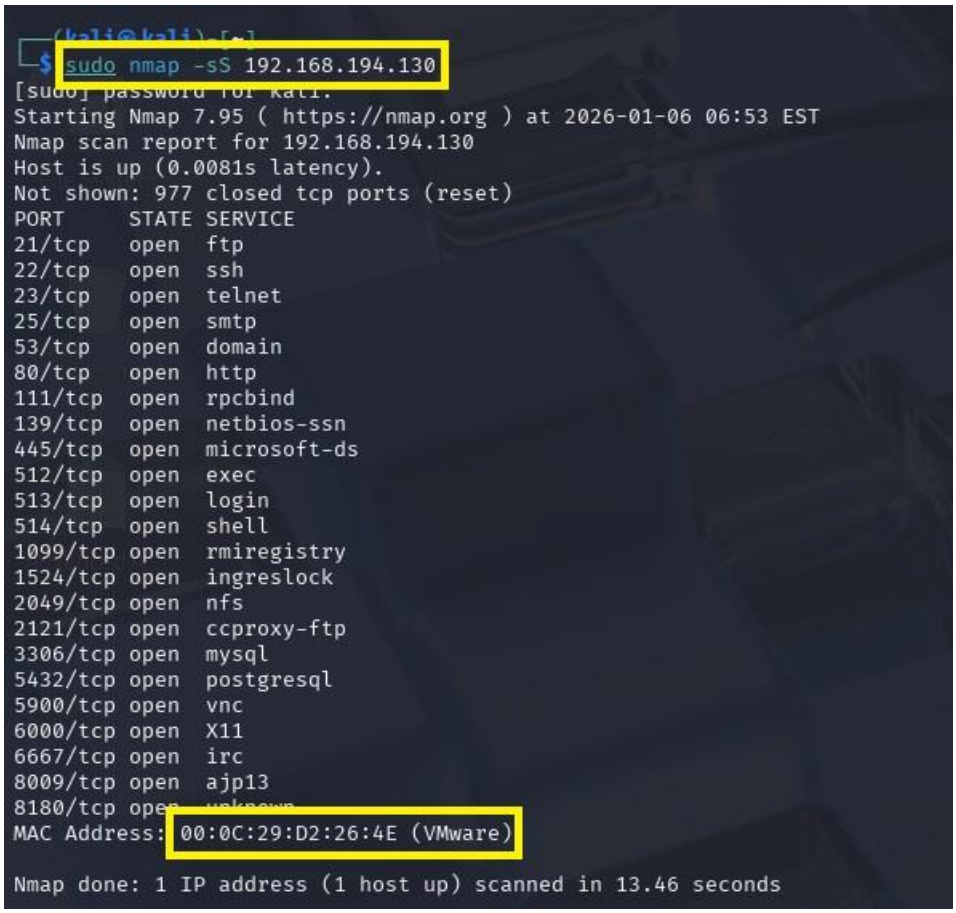


Figure 5.1: TCP SYN scan.

5.2 Service and OS Detection Command:

nmap -A 192.168.194.130

An aggressive scan (-A) was carried out to fingerprint the operating system and service versions following the required 20-minute waiting period. The report correctly identified the web server as Apache 2.2.8 and the target operating system as Linux 2.6.x.

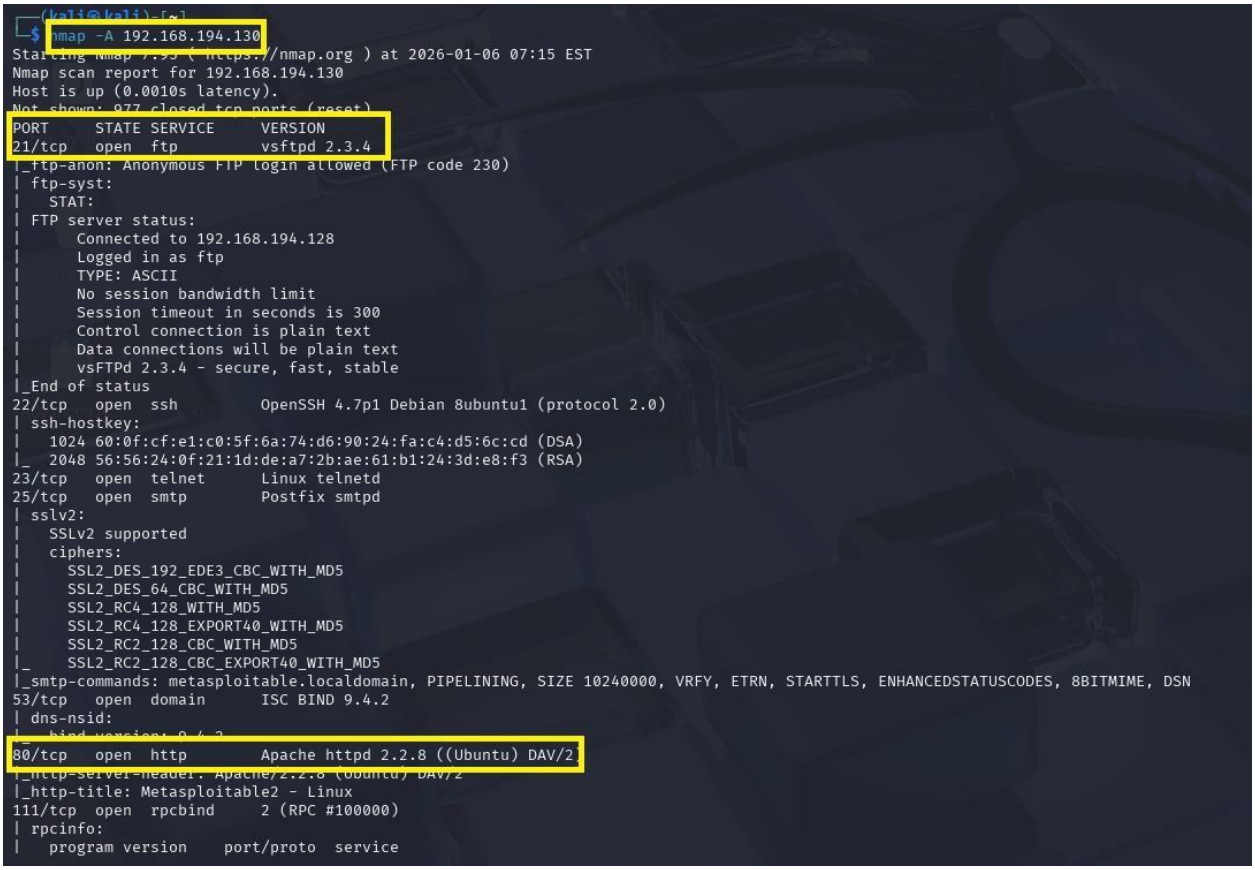


Figure 5.2a: Initiation of Service and OS detection scan.

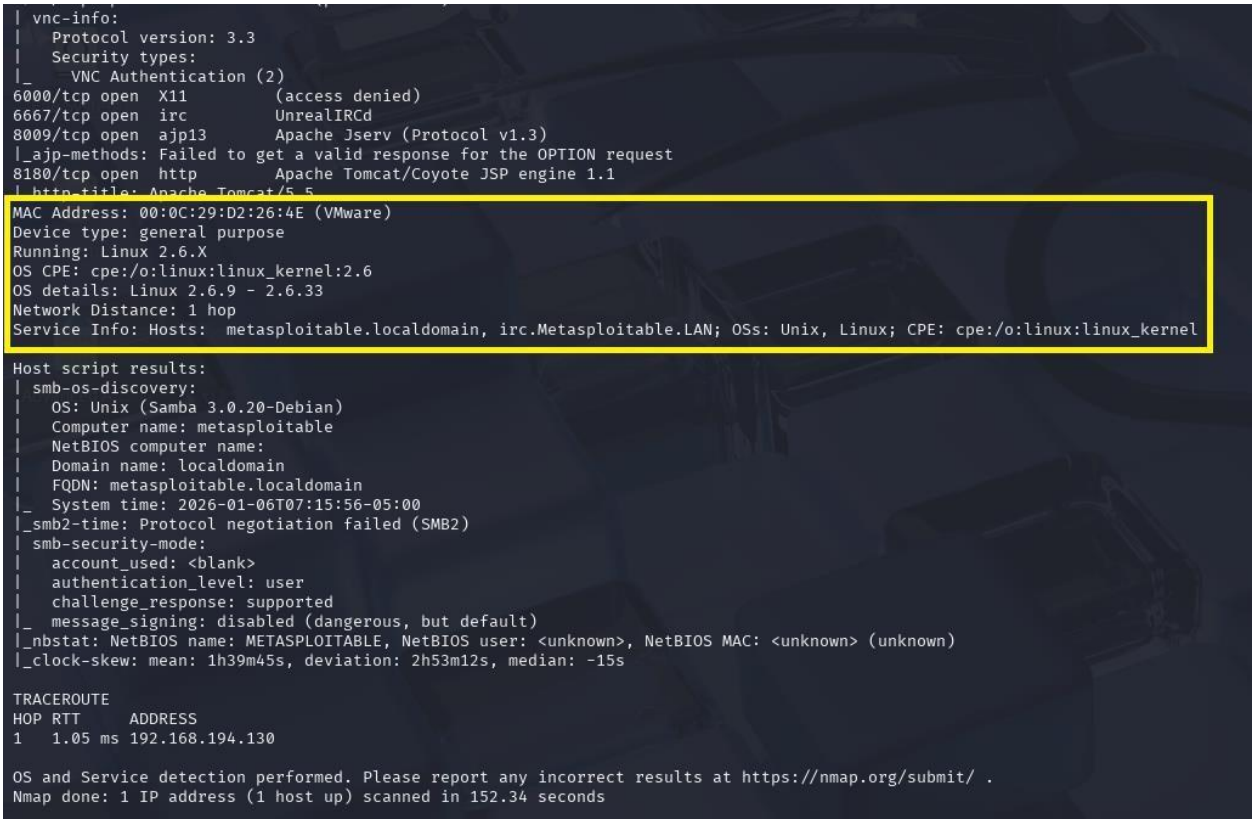


Figure 5.2b: OS (Linux 2.6.9 - 2.6.33) and service versions.

5.3 UDP Port Scan

Command: sudo nmap -sU -F

192.168.194.130

A UDP scan was used to find services that were using non-connection-oriented protocols. This exposed possible attack vectors that are frequently overlooked by routine TCP scans.

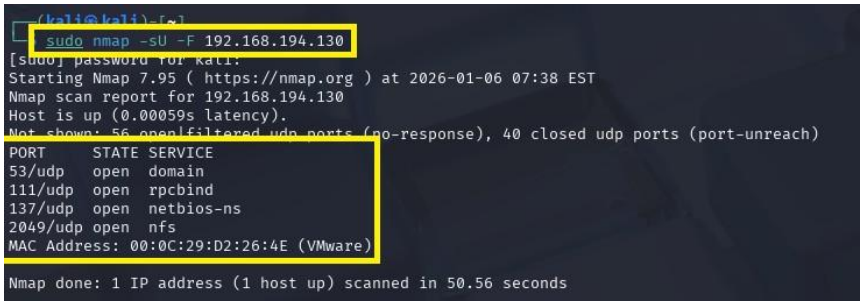


Figure 5.3: UDP scan.

5.4 Web Server Vulnerability Scanning

Command:

nikto -h 192.168.194.130

To find application-layer vulnerabilities, the Nikto web scanner was used. A browsable /doc/ directory, an out-of-date Apache version (2.2.8), and the existence of phpMyAdmin were among the severe concerns found by the scan.

Figure 5.4: Nikto vulnerability

5.5 Denial of Service Attack (Slowloris)

Tool: Metasploit Framework (auxiliary/dos/http/slowloris)

In order to assess the target's resistance to resource depletion, a DoS attack was initiated. The attack used up all of the server's available sockets by opening 150 half-open connections at once.

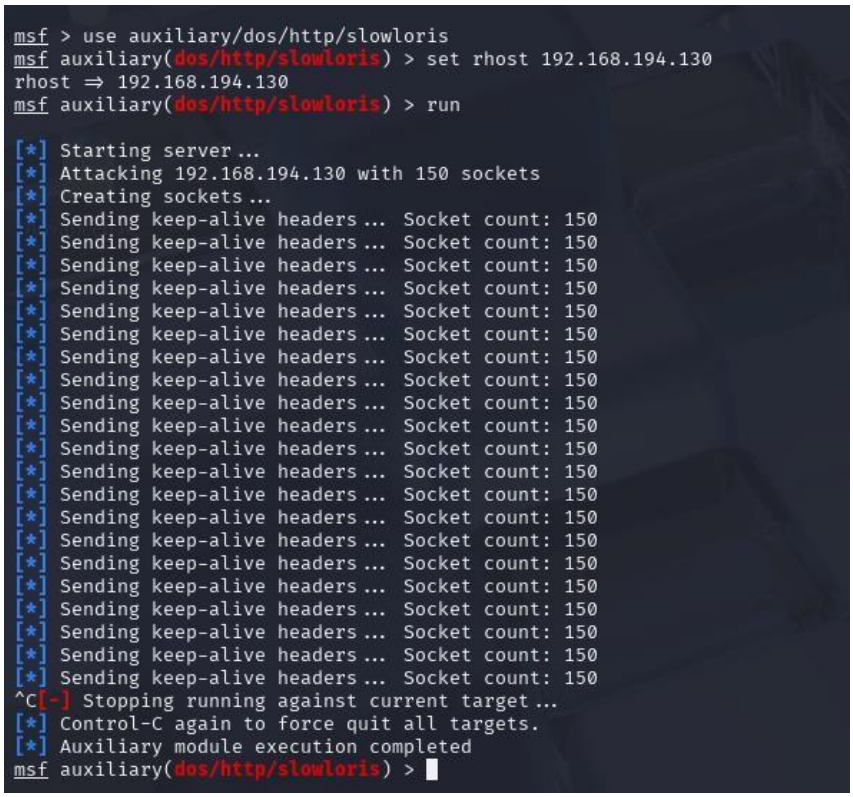


Figure 5.5a: Slowloris attack module.

Impact Analysis:

The attack successfully depleted the server's resources, making the web service inaccessible to authorized users, as seen by the connection count on Port 80 spiking to 152.

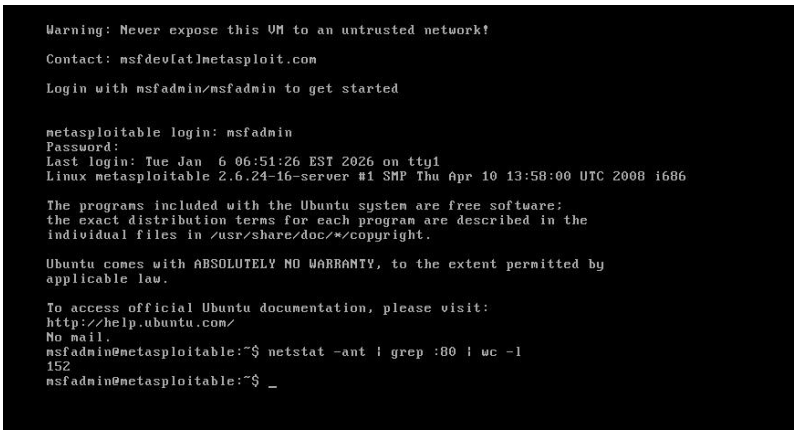


Figure 5.5b: Target system metrics showing resource exhaustion (152 connections) during the attack.

Task 6: Analysis of Log Files and Network Traffic

Following the data collection phase, a forensic analysis was conducted on the captured network traffic (final_project.pcap) and server logs (final_access.log). The objective was to detect patterns, anomalies, and specific attack signatures that corroborate the active reconnaissance and Denial of Service activities performed in Task 5.

6.1 Traffic Pattern and Timeline Analysis

Tool Used: Wireshark I/O Graphs

To see the attack chronology, a timeline analysis of network traffic volume was carried out. The graph below shows unmistakable "spikes" of activity that correspond to the attacker's scans, with significant stretches of inactivity (flat lines) in between.

- Observation: The 20-minute intervals between activity surges are consistent with the timetable.
- Anomalies Detected:
    - Phase 1 & 2 (0s - 1500s, SYN and OS Detection): Sharp, short-duration spikes indicate automated port scanning. o Phase 3 (4200s, Nikto): A denser cluster of traffic corresponds to the web vulnerability scan.
    - Phase 4 (End of Capture): The Slowloris Denial of Service attack is distinguished from the fleeting scanning activity by a persistent, solid block of high-volume traffic.
- Correlation Verification: The attack sequence is verified by cross-referencing the timestamps. The first appearance of the Nikto User-Agent in the Apache logs at 08:04:03 EST corresponds exactly with the web scan spike seen in the Wireshark graph at approximately 4200 seconds, indicating that the network spike was, in fact, HTTP-based vulnerability scanning.

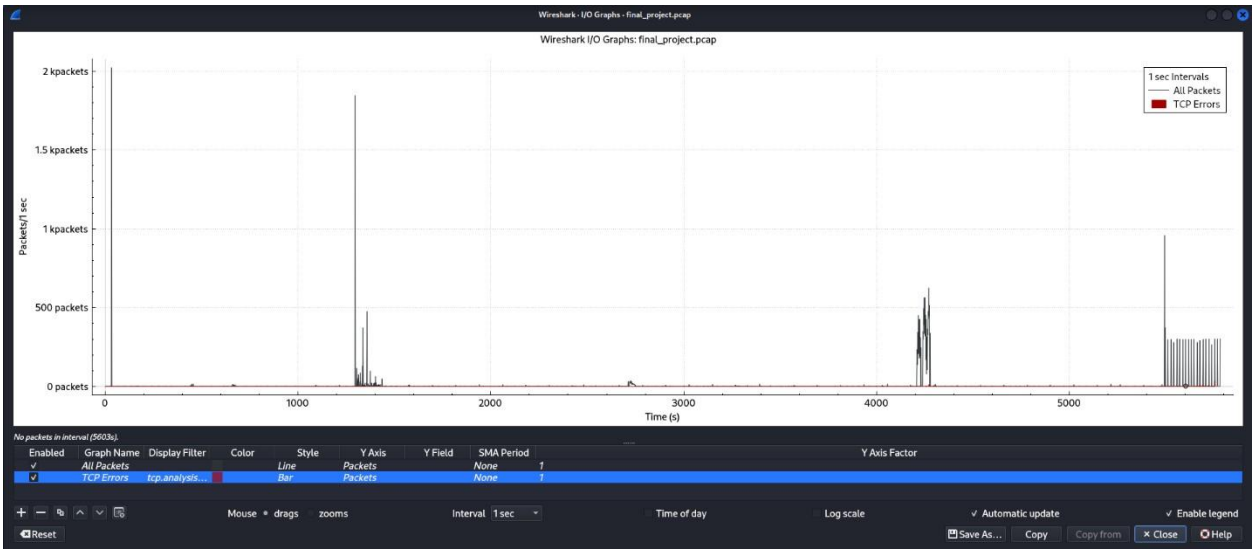


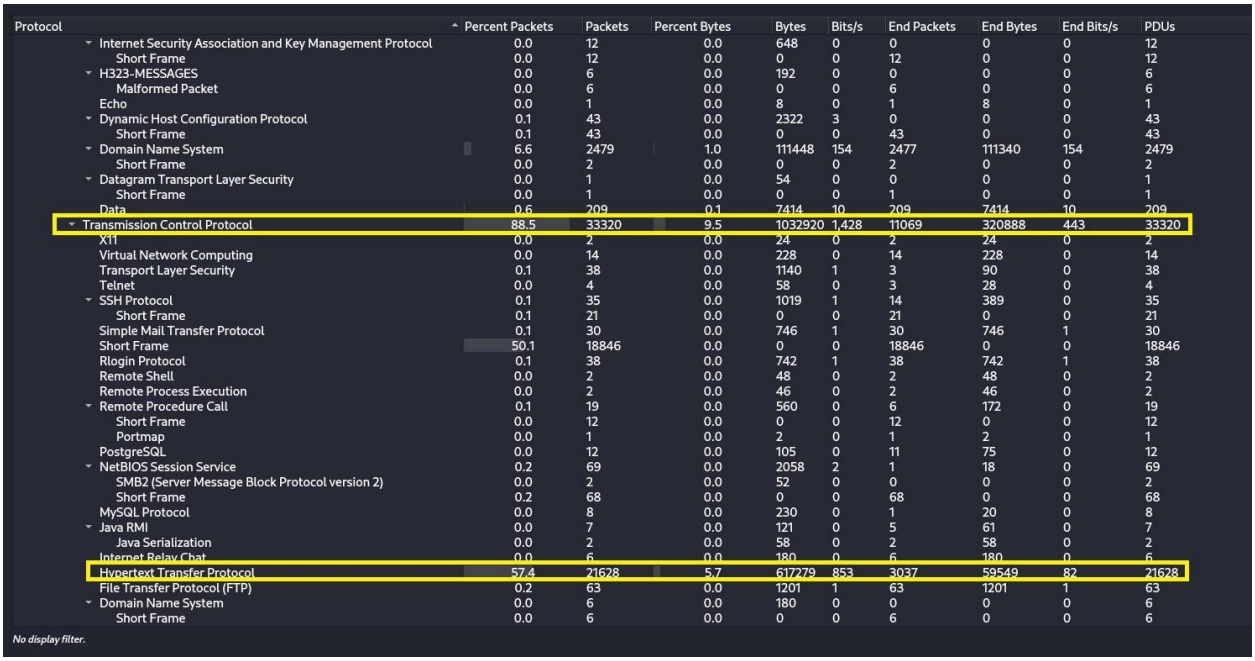
Figure 6.1: Network traffic timeline showing distinct attack phases separated by 20-minute intervals of inactivity.

6.2 Protocol Distribution Analysis

Tool Used: Wireshark Protocol Hierarchy

A protocol hierarchy analysis was carried out to confirm the variety of attack paths. This analysis reveals the precise techniques employed to probe the target by breaking down the collected traffic by protocol type.

- TCP (Transmission Control Protocol): Indicates the dependence on TCP-based scans (SYN, Connect) and the Slowloris attack, accounting for 88.5% of all packets.
- UDP (User Datagram Protocol): A UDP scan or UDP probing activity is consistent with the presence of 3,491 UDP packets (9.3% of traffic), as this amount of UDP data is rarely generated by regular online traffic.
- HTTP (Hypertext Transfer Protocol): Makes up more than 57% of the traffic, indicating that the application layer (Web Server) was the target of the majority of the assault volume.



Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Internet Security Association and Key Management Protocol	0.0	12	0.0	648	0	0	0	0	12
Short Frame	0.0	12	0.0	0	0	12	0	0	12
H323-MESSAGES	0.0	6	0.0	192	0	0	0	0	6
Malformed Packet	0.0	6	0.0	0	0	6	0	0	6
Echo	0.0	1	0.0	8	0	1	8	0	1
Dynamic Host Configuration Protocol	0.1	43	0.0	2322	3	0	0	0	43
Short Frame	0.1	43	0.0	0	0	43	0	0	43
Domain Name System	6.6	2479	1.0	111448	154	2477	111340	154	2479
Short Frame	0.0	2	0.0	0	0	2	0	0	2
Datagram Transport Layer Security	0.0	1	0.0	54	0	0	0	0	1
Short Frame	0.0	1	0.0	0	0	1	0	0	1
Data	0.6	209	0.1	7414	10	209	7414	10	209
Transmission Control Protocol	88.5	33320	9.5	1032920	1,428	11069	320888	443	33320
X11	0.0	2	0.0	24	0	2	24	0	2
Virtual Network Computing	0.0	14	0.0	228	0	14	228	0	14
Transport Layer Security	0.1	38	0.0	1140	1	3	90	0	38
Telnet	0.0	4	0.0	58	0	3	28	0	4
SSH Protocol	0.1	35	0.0	1019	1	14	389	0	35
Short Frame	0.1	21	0.0	0	0	21	0	0	21
Simple Mail Transfer Protocol	0.1	30	0.0	746	1	30	746	1	30
Short Frame	50.1	18846	0.0	0	0	18846	0	0	18846
Rlogin Protocol	0.1	38	0.0	742	1	38	742	1	38
Remote Shell	0.0	2	0.0	48	0	2	48	0	2
Remote Process Execution	0.0	2	0.0	46	0	2	46	0	2
Remote Procedure Call	0.1	19	0.0	560	0	6	172	0	19
Short Frame	0.0	12	0.0	0	0	12	0	0	12
Portmap	0.0	1	0.0	2	0	1	2	0	1
PostgreSQL	0.0	12	0.0	105	0	11	75	0	12
NetBIOS Session Service	0.2	69	0.0	2058	2	1	18	0	69
SMB2 (Server Message Block Protocol version 2)	0.0	2	0.0	52	0	0	0	0	2
Short Frame	0.2	68	0.0	0	0	68	0	0	68
MySQL Protocol	0.0	8	0.0	230	0	1	20	0	8
Java RMI	0.0	7	0.0	121	0	5	61	0	7
Java Serialization	0.0	2	0.0	58	0	2	58	0	2
Internet Relay Chat	0.0	6	0.0	180	0	6	180	0	6
Hypertext Transfer Protocol	57.4	21628	5.7	617279	853	3037	59549	82	21628
File Transfer Protocol (FTP)	0.2	63	0.0	1201	1	63	1201	1	63
Domain Name System	0.0	6	0.0	180	0	0	0	0	6
Short Frame	0.0	6	0.0	0	0	6	0	0	6

No display filter.

Figure 6.2: Protocol hierarchy statistics confirming the presence of TCP, UDP, and HTTP traffic vectors.

6.3 Web Server Log Forensics & Access Patterns

Tool Used: grep, awk (Linux Command Line)

I looked for statistical anomalies and text-based signatures in the Apache server logs (final_access.log).

A. User-Agent Detection (Nikto)

Certain "User-Agent" strings are frequently left by automated scanners. The employment of an automated vulnerability scanner was verified by filtering the logs for "Nikto".

B. Access Pattern Analysis (Targeted Enumeration)

Finding administrative interfaces is evidently the goal, according to an examination of the most often requested resource pathways. The most frequently requested directories were /cgi-bin/, /phpmoadmin/, and /wu-moadmin/. Instead of typical user browsing, this suggests a deliberate attempt to hack database and server administration tools.

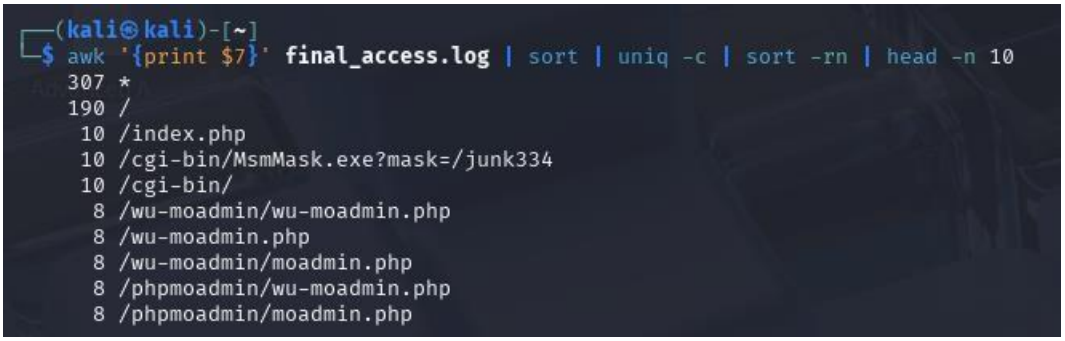


Figure 6.3a: Top 10 requested paths showing repeated attempts to locate administrative panels.

C. Error Code Distribution

The log analysis reveals a significant disparity between requests that are unsuccessful and those that are successful. There were 17,182 "404 Not Found" failures and 4,872 "200 OK" responses in the logs. A "brute-force" directory enumeration attack, in which the scanner guesses thousands of nonexistent file paths, is characterized by this high failure rate (~3.5 failures per success). During the DoS phase, 356 "400 Bad Request" errors were also noted, which is consistent with irregular or incomplete HTTP requests that are frequently produced under connection stress.

Figure 6.3b: HTTP Status Code distribution showing a high volume of 404 errors (enumeration) and 400 errors (DoS).

6.4 NetFlow Analysis

Tool Used: SiLK Toolset (rwptoflow, rwstats)

The rwptoflow software was used to convert the packet capture to SiLK flow records in order to verify the results at the flow level. The most important communication channels by volume were then determined by analyzing the traffic using rwstats.

Observation: With 53.2% of all flow data (19,664 flows), the analysis verifies that the attacker (192.168.194.128) was the main initiator of network events.

The destination port analysis reveals two distinct attack vectors:

1. **TCP Port 80 (HTTP):** With 13,487 records (36.5% of the total), Port 80 had the highest volume of traffic. This large number of unique flows is in line with the Nikto scanner's quick queries and the Slowloris DoS attack's socket-exhaustion behavior.
2. **UDP Port 53 (DNS):** The UDP reconnaissance phase was verified forensically by observing a substantial amount of 2,476 records on UDP Port 53.

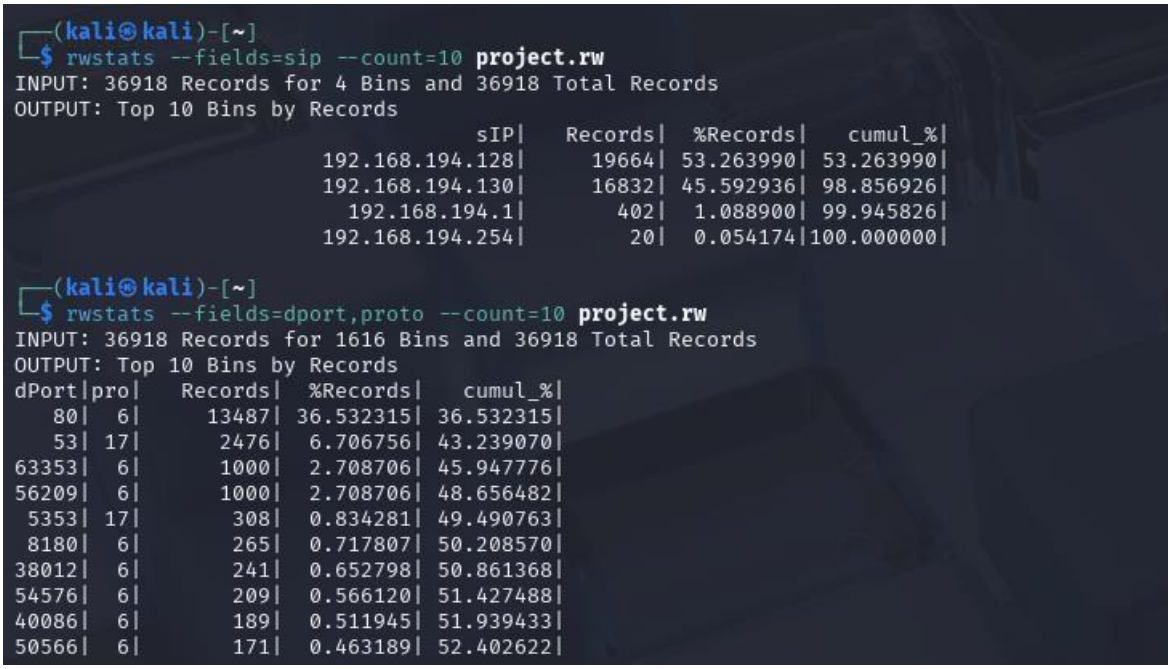


Figure 6.4: SiLK rwstat

Task 2: AI-Driven Defence and Analysis Techniques (group task)

1.Researcher Analysts

Technique 1: LLM-based Log & Alert Analysis

Description:

This technique uses Large Language Models (LLMs) to analyse system and security logs which are used to identify suspicious behaviour or potential attacks. The model does not rely only on fixed rules or statistical thresholds.The model reads log messages as text to understand their meaning and context across multiple log entries.

Traditional intrusion detection systems often generate many alerts because they depend on predefined rules or numerical patterns. These may cause systems to fail to recognise complex attacks or produce many false positives. However, an LLM may interpret the relationships between different log messages such as repeated access attempts, abnormal request sequences or unusual system behaviour.These can help security analysts identify real threats more efficiently.

In a typical LLM-based log analysis framework, historical log data is first prepared and labelled with corresponding outcomes and causes. The language model is adjusted using lightweight techniques such as parameter-efficient fine-tuning to adapt it to security log analysis without requiring extensive computational resources. Once deployed, the model analyses incoming logs and classifies their risk level.The model also provides short and human-readable explanations to describe why an event is considered suspicious or normal.

To improve accuracy, feedback from security analysts can be stored in a knowledge base. When the similar events appear again,the system can refer to previous decisions and corrections.These will allow the analysis process to improve over time. This approach reduces the workload of security operation centre (SOC) staff and supports faster and more informed decision-making during incident response.

Architectural Illustration

To support effective security monitoring and alert analysis, the system operates through a structured pipeline:

1. Log Collection Layer: Collects system and security logs from servers, applications and network devices.
2. Preprocessing and Sanitisation: Extracting useful fields, removing noise and sanitising sensitive information to clean log data.
3. LLM Analysis Core: Uses the LLM to analyse log messages and detect abnormal patterns across multiple log entries.
4. Risk Classification and Explanation: Classifies events as normal or suspicious. Provides simple explanations for the decisions.
5. Alert Presentation to Analysts: Presents analysed alerts and risk levels to security analysts for review.

6. Feedback and Knowledge Update: Stores analyst feedback to improve future log and alert analysis.

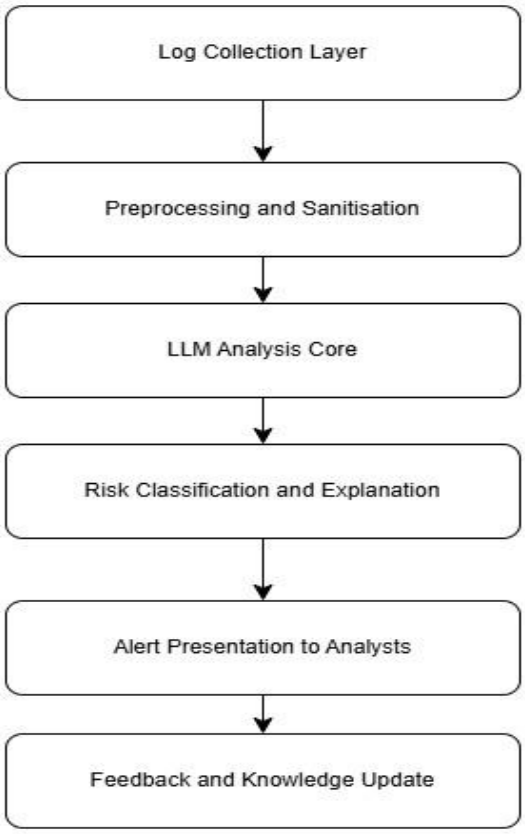


Figure: Architecture of the LLM-based Log & Alert Analysis System

Advantages

- Understands the semantic meaning and context of log messages
- Detects complex or previously unseen attack patterns
- Reduces false positives compared to rule-based IDS
- Analyses multiple related log entries instead of isolated alerts
- Provides clear explanations to support analyst investigation
- Reduces manual workload for SOC analysts
- Supports faster and more informed incident response

Limitations

- May occasionally produce incorrect or misleading interpretations
- Requires human validation for critical security decisions
- Introduces additional computational cost and processing latency
- Sensitive log data must be handled carefully to protect privacy
- Model performance may degrade as system behaviour or log formats change
- Requires ongoing monitoring and maintenance

Technique 2: LLM-Based Automated Threat Intelligence Synthesis

Description

This method automates the ingestion and synthesis of different types of threat intelligence databy utilizing LLMs. This approach, in contrast to the old-fashioned system of keywords, is based on Natural Language Understanding (NLU) that is used to process unstructured resources: scraping forums on the dark web, vendor security blogs, and CVE (Common Vulnerabilities and Exposures) databases.

The model allows a Researcher Analyst to do Named Entity Recognition (extracting IPs, hashes, and malware families) and Relationship Mapping (connecting a particular exploit technique to a familiar group of Threat Actors).

By correlating vast amounts of diverse threat intelligence data, this method greatly minimizes the manual threat hunting effort required by researcher analysts and facilitates the quick production of hypotheses.

Architectural Illustration

To guarantee data integrity, the system runs via a structured pipeline:

1. Data Ingestion Layer: Compiles unprocessed text from API logs, PDF reports, and RSS feeds.
2. Preprocessing and Embedding: Produces vector representations that capture the meaning of the text.
3. LLM Processing Core: The model maps patterns against the MITRE ATT&CK framework, such as TTPs (Tactics, Techniques, and Procedures).
4. Actionable Intelligence Output: Produces structured threat intelligence outputs that can be shared with security systems.

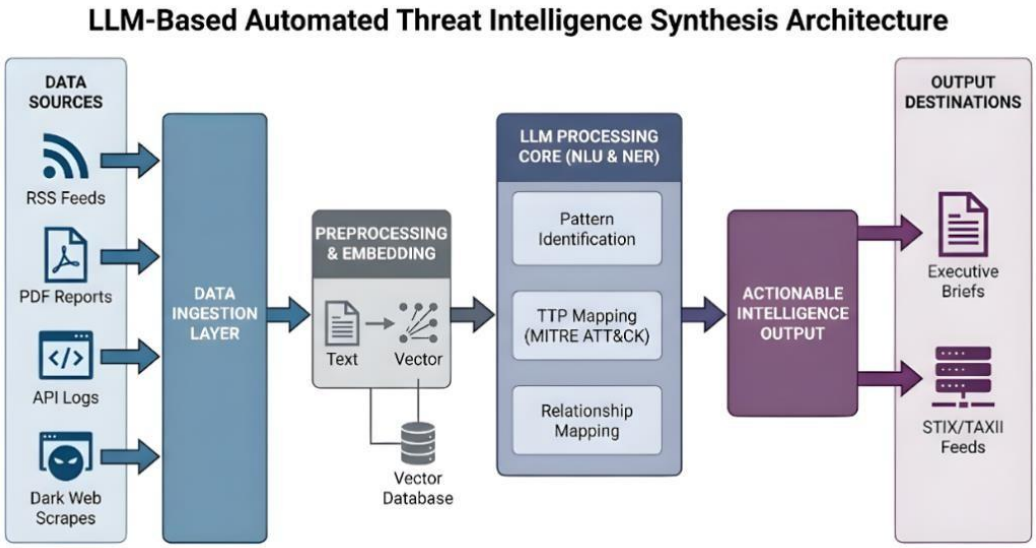


Figure: LLM-Based Automated Threat Intelligence Synthesis workflow

Advantages

- Quick Processing Speed: The model greatly lowers the "Mean Time to Detect" (MTTD) emerging campaign trends by ingesting and synthesizing thousands of pages of security reports in a matter of seconds.
- Semantic Contextualization: LLMs are able to discern between a vulnerability that is being discussed as a theoretical danger and one that is being actively exploited in the wild, unlike basic regex or keyword matching.
- Cross-Language Analysis: Without sacrificing technical context, sophisticated LLMs are able to translate and examine threat alerts from non-English sources (such as Chinese or Russian underground forums).

Limitations

- Risk of Hallucinations: LLMs may occasionally "hallucinate" or produce fake CVE numbers or incorrectly link a malware strain to the wrong threat actor if the training data is inconsistent. Retrieval-Augmented Generation (RAG), which uses human-in-the-loop validation and verified threat information sources, can reduce this risk.
- Knowledge Cut-off Constraints: Unless they are enhanced with Retrieval-Augmented Generation (RAG) to retrieve real-time data, static LLMs are not aware of events that take place beyond their training date.
- Input Token Limits: The model's context window may be exceeded by lengthy, multihundred-page technical whitepapers, necessitating intricate "chunking" techniques that could cause the model to lose the overall context of an attack chain.

Technique 3: LLM-based Incident Response Assistance

Description:

This technique uses Large Language Models (LLMs)to assist security analysts during the incident response process. Incident response often involves analysing alerts, logs, incident reports and historical cases which are used to understand what happened and how to respond. This process can be slow and complex when information is incomplete or scattered across multiple systems.

LLM-based incident response assistance helps analysts to analyse incident details and support root cause analysis. Based on the incident context,LLMs can reason about possible causes and identify relevant patterns from previous incidents which can help to suggest investigation or response steps. When real incident data is limited or cannot be shared due to privacy concerns,LLMs can be trained or supported using synthetic incident response process logs.

When reasoning capabilities are combined with structured incident response knowledge, this technique supports faster and more consistent incident handling and human analysts are responsible for final decisions.

Architectural Illustration

To ensure consistent and reliable incident response assistance,the system operates through a structured workflow:

1. Preparation & Data Collection: Receive alerts and incident information from security systems such as IDS or SIEM.These data sources provide the initial inputs for analysis.
2. Detection & Analysis: The collected data is analysed using multiple analysis perspectives such as source and target correlation, structural analysis, functional analysis and behaviour analysis.The LLM reasons over the incident context to identify abnormal patterns and possible root causes.
3. Alert Prioritization: Based on the analysis results,LLM estimating their severity and potential impact helps them to prioritise alerts. This allows analysts to focus on high-risk incidents.
4. Containment: For prioritize incidents, recommended containment actions are generated to limit damage while awaiting human approval.

5. Eradication & Recovery: The LLM provides proposed mitigation and recovery steps based on past incidents and response guidelines to help analysts during incident resolution.
6. Post-Incident Activity: After the incident is resolved,analyst feedback and incident outcomes are recorded.This information is reused to improve future detection accuracy and response recommendations.

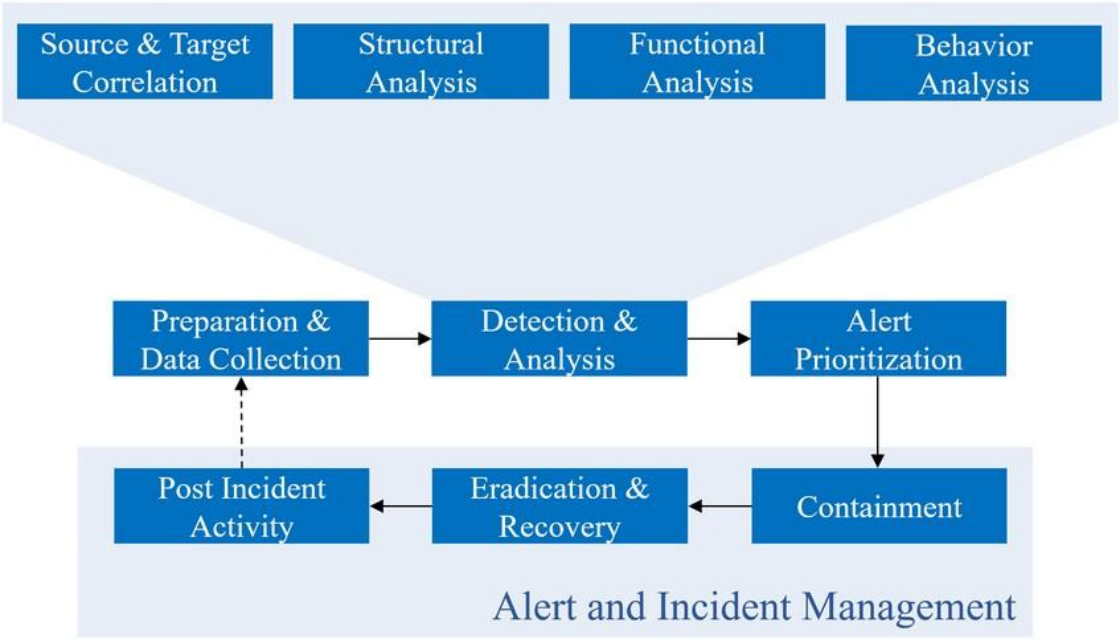


Figure: LLM-Based Incident Response Assistance Workflow

Advantages

- Reduces the time required to analyse and respond to security incidents
- Assists analysts with root cause analysis by reasoning over incident context
- Supports consistent incident handling across different analysts
- Can use synthetic incident response logs when real data is limited
- Helps junior analysts by providing guided response suggestions

Limitations

- LLM recommendations may be inaccurate if incident information is incomplete
- Human validation is required before taking response actions
- Integration with existing incident response tools may be complex
- Synthetic incident logs may not reflect real-world incidents
- Effectiveness depends on the quality of incident data and response knowledge

2.Security Engineer

Within the project team, the Security Engineer is responsible for the technical design, implementation and integration of LLM-based security solutions into XMUMCorp's existing security infrastructure.

This role focuses on ensuring that AI-driven detection mechanisms are operationally effective, secure and scalable. Key responsibilities include evaluating suitable LLM-based techniques, integrating them with systems such as SIEM, Threat Intelligence Platforms, and addressing deployment concerns such as data privacy, model security and performance reliability.

2.1 Comparison of Techniques

Aspect	**LLM-Based Log Analysis**	**LLM-Assisted Threat Intelligence**	**LLM-Based Incident Response Assistance**
Primary Purpose	Detect anomalies and patterns in event logs	Extract threat information from external and internal intelligence sources	Assist and coordinate incident response
Data Sources	SIEM logs, firewall logs, IDS/IPS logs and network logs	Threat feeds, CVE databases and security reports	Alerts from SIEM, EDR, IDS, SOAR

LLM Role	Interpret and summarize massive unstructured log data	Understand threat contexts and derive actionable intelligence	Agent-based decision support and workflow orchestration
Strengths	Reduces manual analysis workload and reveals subtle patterns over time	Improves situational awareness. Supports proactive mitigation	Reduced response time and operational workload
Limitations	Requires extensive preprocessing. May struggle with extremely high log volumes	Susceptible to inaccurate interpretations without manual validation	Risk of over-automation without validation
Security Risks	Data leakage, hallucinations in summaries without context	Misleading threat ratings due to model limitations	Over-automation without human validation

2.2 Applications / Use Cases a.

LLM-Based Log Analysis

- **Anomaly Detection:** Identifies abnormal user or system behaviour that traditional rulebased systems may miss.
- **Incident Triage:** Automatically summarises log events to help SOC analysts quickly understand incidents.
- **Incident Response Support:** Feeds structured insights into SOAR systems to enable faster containment actions.

b. LLM-Assisted Threat Intelligence

- **Threat Aggregation:** Consolidates multiple intelligence sources into actionable insights.
- **Threat Contextualisation:** Maps threats to frameworks such as MITRE ATT&CK.

- **Vulnerability Prioritisation:** Helps security teams focus on vulnerabilities actively exploited in the wild.

c. LLM-Based Incident Response Assistance

- **Response Guidance:** Provides context-aware, step-by-step incident response recommendations.
- **Playbook Automation:** Dynamically generates and adapts incident response playbooks.
- **Post-Incident Reporting:** Automatically produces incident summaries and lessons learned for management and compliance.

2.3 Integration and Deployment Considerations

- **Integration:**

LLM-based log analysis should be layered onto existing SIEM platforms, while threat intelligence analysis integrates with TIPs and vulnerability management systems.

- **Deployment:**

Sensitive log data should be processed on-premise, whereas external threat intelligence can be analysed using cloud or hybrid deployments.

- **Security and Governance:**

LLM systems must be protected against risks such as prompt injection and data leakage through strict access control, data masking, and human-in-the-loop validation.

- **Operational Challenges:**

Continuous model tuning, performance optimisation, and explainability are essential for production deployment.

3. Security Manager

3.1Technology adoption roadmap, training plan and deployment timeline

a. Technology Adoption Roadmap

The deployment of LLM-based security analytics at XMUMCorp will be phased and conservative to maintain operational integrity, data security and visible security outcomes.

Phase 1: Preparation and Risk Assessment (Month 0–2)

- Define high value use cases (such as log analysis, network traffic analysis, DoS detection).
- Perform data sensitivity assessment of logs and network traffic to define handling and storage requirements.
- Compare deployment options (on prem vs private cloud) to reduce data exposure to risk.
- Define security guarantees for deployment (such as false positive rates, MTTR).

Phase 2: Pilot Deployment (Month 3–5)

Deploy LLM in a limited role in the Security Operations Center (SOC) in a read-only, monitoring capacity. Integrate with existing Security Information and Event Management (SIEM) tools for log summarization and anomaly detection explanations. Use existing logged historical PCAP and log data for preliminary deployment. Solicit SOC analyst feedback for improvements.

Phase 3: Operational Deployment (Month 6–9)

Extend LLM deployment to process live network traffic and web server logs. Incorporate Security Orchestration, Automation and Response (SOAR) capabilities for assisted response decision making human review for all high value detections.

Phase 4: Expansion and Maintenance (Month 10–12)

Improve detection strategies and prompts based on operational feedback. Expand the scope of analysis to include other environments (such as cloud services, DNS traffic). Use-case specific performance monitoring and audit reporting.

b. Training plan

A structured training program ensures that personnel can safely and effectively use LLM-based security tools.

Role	**Training Focus**
SOC Analysts	Interpreting LLM outputs, validating alerts, avoiding over-reliance
Security Engineers	Model integration, prompt tuning, system maintenance
Incident Responders	Using LLM-generated summaries for rapid decision-making
Security Managers	Risk interpretation, KPI evaluation, governance oversight
Executives	Strategic value, limitations, and risk acceptance

Training will be conducted through **hands-on SOC simulations**, tabletop exercises, and periodic refresher sessions to address model updates and emerging threats.

c. Deployment timeline

Month	**Key Milestone**
0–2	Risk assessment and use case selection
3–5	Pilot deployment in SOC
6–9	Integration with SIEM/SOAR
10–12	Full operationalization and optimization

3.2 Roles and responsibilities of individuals at various levels within the organization to adopt the technology

Successful adopting the technology requires clear roles at different levels of the organization. This can ensure accountability, security and effective operation.

Executive Management and CISO

Executive management and the CISO play a strategic and governance role in the adoption of LLMbased security tools.They are responsible for approving the security direction, budget and risk level for using LLM-based security tools. They ensure that this technology supports business goals and the organization's security strategy. They are also responsible for high-level governance and longterm risk control.

Security Manager

The Security Manager is responsible for planning and managing the adoption of LLM-based security solutions.This role sets clear goals,monitors risks and ensures the policies are followed. The Security Manager also decides how much automation is allowed and coordinates between management and technical teams.

Security Engineers

Security Engineers are responsible for integrating the LLM into the existing security architecture.They must ensure operational continuity and implement technical safeguards. They must also manage system configurations and enforce access control policies.In addition， their role is to maintain system reliability, performance and security during deployment and operation.

SOC Analysts and Incident Responders

SOC Analysts and Incident Responders are responsible for reviewing LLM-generated alerts and analysis results.Their roles are validating alerts and investigating security incidents.Their roles also include providing feedback to improve model performance and making final response decisions. This is because human judgement is important in high-impact or sensitive incidents.

IT and Compliance Stakeholders

IT and compliance stakeholders play a support and governance role in the adoption of LLM-based security solutions. They are responsible for supporting the deployment, providing infrastructure support and managing system dependencies. They also ensure compliance with internal policies and regulatory requirements.They also support audits, documentation and reporting activities.

3.3 Management, Governance, and Compliance Considerations

Governance

An AI security governance policy will be developed to set acceptable use, decision-making boundaries, and escalation processes. Human-in-the-loop validation will be necessary for all security-related decisions. Model retraining and adjustment processes will adhere to formal change management processes.

Risk Management

To manage risks, analysts will retain ultimate responsibility for decisions to avoid "automation bias". Monitor model drift, bias and inaccurate outputs. No sensitive data will be exposed to external LLM services without approval.

Compliance

All implementations will adhere to relevant data protection legislation such as PDPA and GDPR (where relevant). Audit logs will be created and maintained for all LLM input data, output data and analyst decision making and maintenance. Regular internal review and audits will be performed to ensure compliance.

Ethical and Operational Safeguards

No autonomous enforcement actions will be taken without human approval. Least-privileged access will be applied to all LLM interfaces. Clearly defined accountability for incident handling and decision-making will exist.

Reference

[1] Akhtar, S., Khan, S., & Parkinson, S. (2025, February 2). LLM-based event log analysis techniques: A survey. arXiv.org. https://arxiv.org/abs/2502.00677

[2] Zhang, Z., Li, S., Zhang, L., Ye, J., Hu, C., & Yan, L. (2025). LLM-LADE: Large language model-based log anomaly detection with explanation. *Knowledge-Based Systems*, *326*, 114064. https://doi.org/10.1016/j.knosys.2025.114064

[3] Shafee, S., Bessani, A., & Ferreira, P. M. (2025). False Alarms, Real Damage: Adversarial Attacks Using LLM-based Models on Text-based Cyber Threat Intelligence Systems. ArXiv.org. https://arxiv.org/abs/2507.06252

[4] *Large Language Model (LLM)-Driven Document Clustering: improving Real-Time Security intelligence extraction and Threat analysis*. (2025, July 12). IEEE Conference Publication | IEEE Xplore. https://ieeexplore.ieee.org/abstract/document/11201090

[5] Roy, D., Zhang, X., Bhave, R., Bansal, C., Las-Casas, P., Fonseca, R., & Rajmohan, S. (2024). Exploring LLM-Based Agents for Root Cause Analysis. *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 208–219. https://doi.org/10.1145/3663529.3663841

[6] Galadima, H. S., Doherty, C., & Brennan, R. (2024). Towards LLM-based Synthetic Dataset Generation of Cyber Incident Response Process Logs. *2024 Cyber Research Conference - Ireland (Cyber-RCI)*, 1–4. https://doi.org/10.1109/cyber-rci60769.2024.10939563

[7] Song, S., Zhang, Y., & Gao, N. (2025). Confront Insider Threat: Precise Anomaly Detection in Behavior Logs Based on LLM Fine-Tuning. *ACL Anthology*, 8589–8601. https://aclanthology.org/2025.coling-main.574/

[8] Meng, Y., Tang, L., Yu, F., Jia, J., Yan, G., Yang, P., & Xi, Z. (2025, September 28). *Uncovering vulnerabilities of LLM-Assisted cyber threat intelligence*. arXiv.org. https://arxiv.org/abs/2509.23573

[9] Huo, S., Rong, H., Zhou, Y., Zuo, H., Liu, D., Li, Z., Wang, M., & Jiang, H. (2025). Crash scene to resolution: LLM-based agents driven for efficient traffic accident handling. ACM Transactions on Internet of Things, 6(4), 1–32. https://doi.org/10.1145/3766899

[10] *What is LLM (Large Language Model) Security? | Starter Guide*. (n.d.). Palo Alto Networks. https://www.paloaltonetworks.com/cyberpedia/what-is-llm-security

Team Members and Individual Responsibilities

Name	**Role**	**Responsibilities**
Oh Kai Jun	Researcher Analyst	Developed Technique 1: LLM-based Log and Alert Analysis, Technique 3: LLM based Incident Response Assistance, advantages and limitations. Security Manager: Roles and Responsibilities for Technology Adoption. Consolidated report, managed task assignment.
Md Raiyan Alam Alif	Researcher Analyst	Developed Technique 2: LLM-Based Automated Threat Intelligence Synthesis, advantages and limitations.
Ten Wen Long	Security Engineer	Compared techniques. Identified use cases. Integration and deployment considerations.
Lee Zi Xyuan	Security Manager	Technology roadmap. Training plan. Management, governance and compliance